

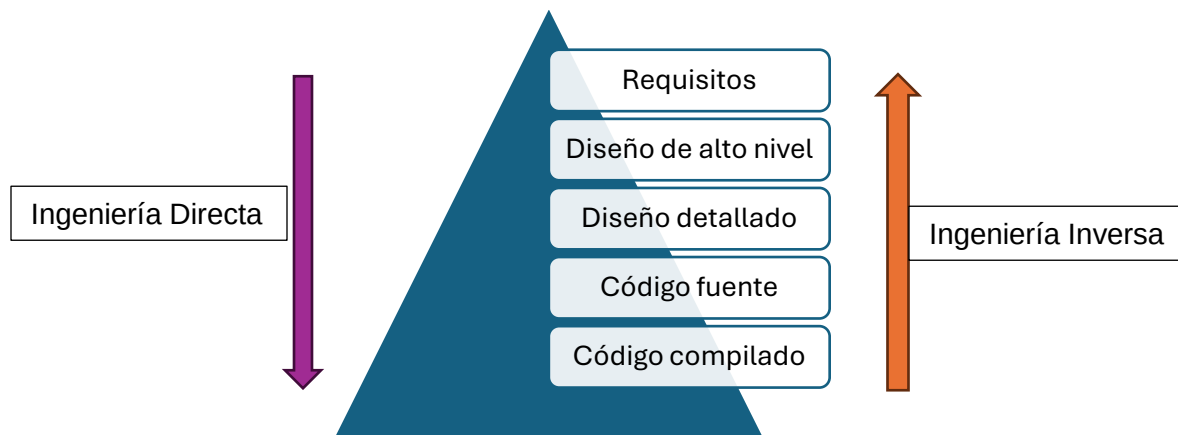
Ingeniería Inversa

Apuntes de Clase

Este material reúne conceptos relacionados con mantenimiento de software, tomados de diferentes fuentes bibliográficas.

La ingeniería inversa, en el contexto del software, es el proceso de analizar un programa con el fin de crear una representación de un **mayor nivel de abstracción**.

En la siguiente figura se representa, de manera muy general, algunos niveles de abstracción de un programa y lo que sería la ingeniería directa y la ingeniería inversa:



Gran parte de la ingeniería inversa está orientada a recuperar el código fuente de un programa a partir del código compilado (es decir, en hexadecimal o binario), que muchas veces es lo único con lo que se cuenta. Para esto se usan herramientas como de-compiladores, desensambladores, depuradores y otros. Hoy en día incluso existe una rama de la ciberseguridad que se dedica a la ingeniería inversa para ver cómo se ha realizado un determinado programa (ya sea para saltarse algún tipo de protección o para intentar descubrir cómo se ha creado un *malware*, por ejemplo).

Sin embargo, la idea se ha extendido, y en el mundo de la ingeniería de software la ingeniería inversa busca recuperar el diseño de un programa a partir del código fuente. Esto es de gran utilidad cuando se desea hacer algún cambio a un programa, pero no hay documentación o está desactualizada.

Recordando los pasos de un mantenimiento, una de las primeras tareas que deben realizar los ingenieros de software antes de responder a cualquier solicitud, es entender el sistema que necesita modificarse, y esta es una de las actividades que más toma tiempo. Por este motivo, la ingeniería inversa es una gran ayuda en esta tarea. Para facilitar aún más esta labor existen varias herramientas que permiten extraer la información del diseño de datos, arquitectónico y detallado de un programa existente. En algunos casos, la primera actividad de ingeniería inversa intenta construir un diagrama de clases de UML (o similar) para entender su organización. A nivel del programa es común que las estructuras de datos internas de este se sometan a ingeniería inversa (como los registros, listas, etc.), y a nivel del sistema es común

que se aplique ingeniería inversa a las estructuras de datos globales (como archivos o bases de datos). Posteriormente se puede tratar de hacer ingeniería inversa a un nivel más detallado para entender la funcionalidad, por ejemplo, mediante diagramas de secuencia, diagramas de bloques (con las funciones o componentes), entre otros. Lo más complicado es tratar de representar las secciones más internas para encontrar patrones de comportamiento.

Aunque las herramientas agilizan el proceso de ingeniería inversa, siempre es importante revisar los resultados y complementar la información con otras fuentes o con inspecciones manuales del código para lograr tener una visión más completa y detallada del software que se desea estudiar, lo cual se conoce como un enfoque semiautomatizado.

En general se tienen tres pasos cuando se trabaja ingeniería inversa:

1. **Obtención de la información:** En este paso se identifican los elementos que tienen los datos más importantes para poder realizar el proceso (por ejemplo, algunas carpetas o archivos en particular, descartando lo que no sea relevante). También se hace un análisis para encontrar una organización preliminar o incluso documentación existente.
2. **Modelado:** elaborar los modelos de diseño a partir de la información seleccionada. Aquí por lo general se hace uso de herramientas.
3. **Revisión:** Verificar si el modelo está completo y representa adecuadamente el programa. También se pueden refinar los modelos con información que tenga el ingeniero o que pueda identificar manualmente.

Existen importantes beneficios que se pueden obtener con la ingeniería inversa, dado que contribuye, entre otros, a:

- Reducir los costos de entender un programa cuando se desea hacer un mantenimiento.
- Realizar mejoras en un programa existente, especialmente para adaptarse a nuevas tecnologías.
- Restructurar o transformar un sistema para que tenga un mejor diseño o arquitectura.
- Identificar posibles vulnerabilidades en un sistema.
- Entender y documentar sistemas complejos.
- Generar vistas alternativas para un sistema.
- Identificar posibles efectos secundarios cuando se realice un cambio.

Sin embargo, también la ingeniería inversa tiene un aspecto ético y legal que debe considerarse, por ejemplo, cuando se trata de utilizar en software sobre el cual no se tienen los derechos o permisos correspondientes, por lo que siempre se debe aplicar un criterio juicioso para decidir si es pertinente su aplicación.

Referencias bibliográficas

- Baeldung, Eugen. "What Is Reverse Engineering in Computer Science?", de Baeldung, 2023. Disponible en: <https://www.baeldung.com/cs/reverse-engineering>
- Lutkevich, Ben (TechTarget). "What is reverse-engineering?", 2021. Disponible en: <https://www.techtarget.com/searchsoftwarequality/definition/reverse-engineering>
- Serra Ruiz, Jordi. "Ingeniería inversa: qué es, herramientas y técnicas", 2021. Disponible en: <https://blogs.uoc.edu/informatica/es/ingenieria-inversa-que-es-herramientas-y-tecnicas/>